
Exercícios: Variáveis em C

Thalles Eduardo Silva Mota

10 de abril de 2026

Resumo

Este trabalho apresenta duas soluções para o problema clássico de troca de valores entre duas variáveis inteiras na linguagem C. A primeira abordagem utiliza uma variável auxiliar temporária para preservar um dos valores durante a permutação. A segunda abordagem realiza a troca exclusivamente por meio de operações aritméticas de soma e subtração, dispensando memória adicional. Ambas as implementações demonstram conceitos fundamentais de manipulação de variáveis, atribuição e expressões aritméticas em programação estruturada.

Palavras-chave: troca de variáveis; variável auxiliar; operações aritméticas; linguagem C; atribuição .

Introdução

A manipulação de variáveis constitui uma das operações mais elementares no estudo de programação de computadores. Compreender como os dados são armazenados, acessados e modificados na memória é um passo essencial para o desenvolvimento do raciocínio lógico-computacional exigido na formação em Ciência da Computação.

Dentre os problemas introdutórios frequentemente propostos, a troca de valores entre duas variáveis ocupa posição de destaque por sua simplicidade aparente e pela riqueza conceitual que oferece. Embora o enunciado seja direto — permutar o conteúdo (trocar os valores) de duas variáveis inteiras — sua resolução mobiliza conceitos fundamentais como declaração de variáveis, comando de atribuição, uso de variáveis auxiliares e construção de expressões aritméticas.

O presente trabalho aborda duas estratégias distintas para a solução desse problema na linguagem C. A primeira emprega uma variável temporária como espaço intermediário de armazenamento, garantindo que nenhum valor seja perdido durante o processo de troca. A segunda estratégia elimina a necessidade de memória adicional ao explorar propriedades das operações de soma e subtração para rearranjar os valores utilizando apenas as duas variáveis originais.

O objetivo é não apenas apresentar as implementações, mas também discutir as vantagens, limitações e os conceitos de programação envolvidos em cada abordagem, contribuindo para a consolidação dos fundamentos de algoritmos em uma programação computacional estruturada.

Estratégia 1 - Troca com variável auxiliar

A primeira solução utiliza uma variável auxiliar temporária chamada *aux* para intermediar a troca. O processo ocorre em três passos sequenciais: primeiro, o valor de *a* é copiado para *temp*; em seguida, *a* recebe o valor de *b*; por fim, *b* recebe o valor armazenado em *aux*, que corresponde ao valor original de *a*.

Essa abordagem é considerada o método padrão de troca de variáveis por sua simplicidade e confiabilidade. O custo adicional de memória é mínimo — apenas uma variável inteira extra — e não há risco de erros numéricos, independentemente dos valores envolvidos. Tais conceitos, são demonstrados na seção 2.1.3 do livro citado:[Medina e Fertig 2006].

O Código 1 apresenta a implementação completa desta solução.

Código 1: Troca com variável auxiliar

```
1  /* =====
2  Estrategia:
3  - Usa uma terceira variavel (aux) para guardar
4    temporariamente o valor de 'a' antes de sobrescreve-lo.
5  - Passo 1: aux recebe o valor de a
6  - Passo 2: a recebe o valor de b (a antigo esa salvo)
7  - Passo 3: b recebe o valor de aux (que e o a antigo)
8  ===== */
9
10 #include <stdio.h>
11
12 int main() {
13     int a, b, aux;
14
15     printf("Digite dois inteiros: ");
16     scanf("%d %d", &a, &b);
17
18     printf("Antes: a = %d, b = %d\n", a, b);
19
20     aux = a;
21     a = b;
22     b = aux;
23
24     printf("Depois: a = %d, b = %d\n", a, b);
25
26     return 0;
27 }
```

Estratégia 2 - Troca sem variável auxiliar

A segunda solução elimina a necessidade de uma terceira variável ao explorar uma propriedade das operações de soma e subtração: se *a* recebe $a + b$, então $a - b$ recupera o valor original de *a*, e uma segunda subtração recupera o valor original de *b*.

Concretamente, o processo se dá em três passos: $a = a + b$ faz com que *a* passe a conter a soma dos dois valores; $b = a - b$ atribui a *b*. Ao final, os valores estão trocados

sem uso de memória adicional.

Embora elegante, essa técnica apresenta uma limitação prática: se a soma de a e b ultrapassar o limite do tipo *int* (aproximadamente $\pm 2,15 \times 10^9$ em arquiteturas de 32 bits), ocorrerá *overflow*, produzindo resultados incorretos. Portanto, sua solução está proposta no seguinte na seção 1.3 do seguinte livro: [Knuth 1997].

O Código 2 apresenta a implementação completa desta solução.

Código 2: Troca sem variável auxiliar

```
1  /* =====
2  Estrategia:
3  - Usa apenas operacoes de soma e subtracao sobre
4  as proprias variaveis a e b.
5  - Passo 1: a = a + b -> a agora guarda a soma dos dois
6  - Passo 2: b = a - b -> subtrai b da soma, restando
7  o valor original de a
8  - Passo 3: a = a - b -> subtrai o novo b (antigo a)
9  da soma, restando o valor
10 original de b
11
12 Exemplo com a=5, b=3:
13   a = 5+3 = 8
14   b = 8-3 = 5 (valor original de a)
15   a = 8-5 = 3 (valor original de b)
16 ===== */
17 #include <stdio.h>
18
19 int main() {
20     int a, b;
21
22     printf("Digite dois inteiros: ");
23     scanf("%d %d", &a, &b);
24
25     printf("Antes: a = %d, b = %d\n", a, b);
26
27     a = a + b;
28     b = a - b;
29     a = a - b;
30
31     printf("Depois: a = %d, b = %d\n", a, b);
32
33     return 0;
34 }
```

Conclusão

O presente trabalho abordou o problema clássico de troca de valores entre duas variáveis inteiras na linguagem C, apresentando duas soluções distintas que, embora conduzam ao mesmo resultado, diferem em estratégia algorítmica e nos conceitos que mobilizam.

A primeira solução, baseada em uma variável auxiliar, destaca-se pela clareza e segurança. Sua lógica é intuitiva e de fácil compreensão, o que a torna a abordagem recomendada na maioria dos cenários práticos. A introdução de uma terceira variável como espaço temporário de armazenamento ilustra de forma didática o funcionamento do comando de atribuição e o princípio de que uma atribuição sobrescreve o valor anterior de uma variável.

A segunda solução, que dispensa memória adicional ao utilizar exclusivamente operações de soma e subtração, demonstra como propriedades aritméticas podem ser exploradas para resolver problemas de manipulação de dados. Embora elegante, essa abordagem apresenta a limitação de possível "overflow" quando os valores envolvidos são muito grandes, o que deve ser considerado em aplicações reais.

Ambas as implementações reforçam conceitos fundamentais trabalhados na disciplina de Algoritmos I, como declaração de variáveis, operador de atribuição, entrada e saída de dados com "scanf" e "printf", e construção de expressões aritméticas. A análise comparativa das duas abordagens contribui para o desenvolvimento do pensamento computacional, evidenciando que um mesmo problema pode admitir múltiplas soluções com diferentes compromissos entre simplicidade, eficiência e robustez.

Referências

- [Knuth 1997] KNUTH, D. E. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. 3. ed. [S.l.]: Addison-Wesley, 1997.
- [Medina e Fertig 2006] MEDINA, M.; FERTIG, C. *Algoritmos e Programação: Teoria e Prática*. 2. ed. São Paulo: Novatec, 2006.